

03-26 Лекция: скиллы агентов, маршрутизация задач и стек голосовых технологий (ASR/STT, TTS, клонирование, live модели)

Дата и время: 2026-03-26 10:50:58

Локация: [Вставить локацию]

Инструктор: [Вставить имя]

Резюме

Лекция охватывает концепцию скиллов для агентных систем, практики маршрутизации задач и экономии ресурсов контекста, а также полный стек голосовых технологий: распознавание речи (ASR/STT), транскрибацию, синтез речи (TTS), клонирование голоса, интеграцию в ботов и переход от классических конвейеров ASR→LLM→TTS к live/end-to-end голосовым моделям. Показано, как скиллы заменяют громоздкие промпты и повышают точность через дерево задач, почему это важно для локальных моделей с ограничениями VRAM. В блоке про речь детально разбираются Whisper и Faster Whisper, провайдеры (Deepgram и др.), метрики скорости/стоимости, проблемы диаризации, числительных и имен собственных. Рассмотрены бесплатные и платные TTS-решения (Edge TTS, Google AI Studio, ElevenLabs), качество, стоимость (ориентир ~2 цента/мин), управление стилем/интонацией, multi-speaker сценарии и правовые аспекты. Обсуждается рыночный сдвиг от «просто ASR» к комплексным голосовым агентам, интеграции через API и архитектурные компромиссы: латентность и ненадежность разнесенного конвейера против преимуществ live моделей с потоковой генерацией и движением к дуплексному диалогу.

Ключевые пункты знаний

1. Скиллы как базовый строительный блок агентных систем

- Скилл — модуль с инструкциями и вспомогательными файлами (скрипты, шаблоны, референсы), упакованный в директорию с обязательным

[Skill.md](#). Краткое однострочное описание в [Skill.md](#) используется для каталогизации и вызова скилла.

- Агенты читают [Skill.md](#), по требованию подгружают нужные файлы и исполняют код, вместо включения всего в системный промпт.
- Вынесение повторяющихся инструкций в скиллы сокращает контекст, экономит токены и VRAM, особенно в локальном запуске моделей.
- В экосистеме имеется «skill-creator» как эталон стандартов, упрощающий создание и унификацию скиллов.

2. Скиллы vs длинные промпты: ресурсы и качество

- Длинные промпты (десятки страниц) раздувают контекст, вынуждая использовать меньшие модели из-за VRAM и снижая качество.
- Практический вывод: монолитные промпты в задачах аудита звонков или CV приводят к нерелевантностям и слабой ресурсоэффективности.
- Рекомендация: дробить задачи на дерево категорий и запускать узкие скиллы последовательно. Это повышает точность и снижает нагрузку на память.

3. Маршрутизация и дерево задач

- Подход поэтапной детализации: сначала крупные категории, затем переход в поддеревья и запуск узких скиллов.
- Пример CV: вместо однопроходного поиска всех проблем на лице — многоуровневая классификация (морщины, старение, обвислость, акне и т.д.) с поочередным уточнением.
- Применение к аудиту звонков: разделение на проверку скрипта, выявление ошибок и дополнительные аспекты качества; каждый узел — отдельный скилл.

4. STT/ASR: принципы, модели и ограничения

- STT преобразует аудио в текст; качество зависит от языка, обучающих данных и адаптаций. Сложности: имена собственные, жаргон/OOV, пунктуация, интонация, числительные, диаризация.
- Whisper: открытая многоязычная модель, «веер» размеров (tiny→turbo), работает локально; Faster Whisper ускоряет до ~4×. Извлекает текст и тайм-коды (удобно для субтитров).
- Скорости: локально CPU около 1:1, GPU ~1:10–1:50; VRAM ≥ 6 ГБ желательна. Параллельная транскрибация возможна при достаточном железе.

- Провайдеры и цены: конкуренция по стоимости и скорости; 1,000 минут могут стоить от ~\$0.33 до ~\$6. Важно тестировать качество, не ориентироваться только на цену.
- Deegram: простой старт (API-ключ, Playground), настройки языков/пунктуации/диаризации/форматирования; демонстрация «почти в реальном времени» с разделением спикеров; остаются ошибки на числительных и именах.

5. От ASR к комплексным голосовым агентам

- Рынок смещается от «просто распознавания» к голосовым агентам и end-to-end решениям. Для промышленного применения — специализированные ASR по API; для ситуативных задач — загрузка в Gemini/ChatGPT.
- Интеграция в ботов: обработка голосовых сообщений, отправка аудио на ASR (например, Deegram), возврат текста; можно задать правила мультимодального ответа (голос↔текст).

6. TTS: качество, стоимость и инструменты

- TTS сложнее ASR: нужно управлять интонацией, тембром, темпом, эмоциями, ударениями, акцентами; разнообразие голосов и стилей.
- Edge TTS: бесплатный в разумных пределах, мультязычный; скорость ~2× реального времени; ускорение через параллельные запросы. Возможны огрехи ударений и числительных; подходит для утилитарных задач.
- Google AI Studio: более выразительное TTS, выбор голоса и стиля; режимы single и multi-speaker (диалоги, подкасты), но медленнее и с ограничениями по квотам.
- ElevenLabs: лидирующее качество и консистентность для художественных задач, подписочная модель, дороже; оптимален при требовании стабильного бренд-голоса.
- Стоимость TTS: ориентиры — вход по токенам почти бесплатен (например, ~\$1 за 1,000,000 токенов), расчетно ~2 цента за минуту аудио при некоторых тарифах/моделях; крупные проекты (книги) получаются дешево по сравнению с дикторами. Проверка актуальных прайсов обязательна.

7. Клонирование голоса: качество и безопасность

- Для базового тембра достаточно нескольких секунд ($\approx 3-10$ с), для стабильной интонации — больше данных (минуты). Защитные механизмы провайдеров: проверочный текст/процедуры верификации.
- Локальные эксперименты возможны и быстры; юридические риски при использовании чужих голосов требуют оценки и соблюдения прав.

8. Архитектуры голосового взаимодействия и эволюция к live-моделям

- Классический пайплайн (ASR→LLM→TTS) страдает от накопленной латентности (≈ 2 с и более) и ненадежности из-за множества звеньев и сетевых задержек.
- Оптимизация стримингом (частичная транскрипция в LLM, ранний запуск TTS) снижает задержки, но все еще недостаточно для естественного диалога.
- Тренд: live/end-to-end голосовые модели, где распознавание, языковая обработка и синтез объединены; выдача голосовых токенов в потоковом режиме, без явных промежуточных этапов.
- Симплекс vs дуплекс: сегодняшние системы чаще симплексные (сложно перебивать), цель — полноценный дуплекс (одновременное говорение и слушание). Демонстрации NVIDIA показывают движение к естественному диалогу.

9. Оркестраторы и их ограничения

- Пример WAPI: агрегирует внешние ASR/LLM/TTS и телефонные линии; суммарная стоимость порядка $\sim \$0.11/\text{мин}$ (зависит от выбора), задержки около 2 с; отмечена нестабильность и устаревание подхода на фоне live-моделей.

10. Практические советы по постановке и интеграции задач

- Структурировать голосовые пайплайны как дерево задач и скиллов: экономия контекста, повышение качества, повторная используемость.
- Для субтитров использовать модели с тайм-кодами; для числительных и имен — внедрять постобработчики (нормализация чисел, справочники имен).
- В TTS генерировать «чанками» (абзацы/строфы) для удобной регенерации и контроля интонаций; параллелить запросы для ускорения.
- Выбор инструментов под сценарий: бесплатные студии/Edge TTS для утилитарных задач; ElevenLabs/AI Studio — для художественных и брендовых; учитывать правовые ограничения.

Вопросы

- [Вставить вопрос/непонимание]

Задания

- [] 1. Создать скилл с обязательным [Skill.md](#) (название и однострочное описание), вынести повторяющиеся инструкции из системного промпта и организовать подгрузку скриптов/шаблонов по требованию.
- [] 2. Построить дерево задач для аудита звонков: определить 3–4 верхнеуровневые цели, развернуть подзадачи; настроить последовательный запуск узких скиллов.
- [] 3. Оценить доступные ресурсы (например, 24 ГБ VRAM) и подобрать модель/контекст под отказ от чрезмерных промптов.
- [] 4. Настроить локальный Whisper/Faster Whisper; измерить скорость на CPU/GPU; подготовить тестовый набор аудио с именами/числительными/эмоциями; сравнить провайдеров по качеству и цене (включая Deepgram).
- [] 5. Зарегистрироваться в Deepgram, получить API-ключ, выполнить транскрибацию через Playground и API с пунктуацией и диаризацией; оформить таблицу скоростей/стоимостей/качества.
- [] 6. Реализовать субтитры с тайм-кодами и наложить на видео; подготовить постобработчик для числительных и имен; оценить прирост качества.
- [] 7. Подключить STT к боту: при голосе — отправлять в ASR и возвращать текст; реализовать правило «голос→текст, текст→голос» с TTS.
- [] 8. Установить Edge TTS, сгенерировать русскоязычные фрагменты, настроить скорость/тембр; провести эксперимент с параллельной генерацией; задокументировать качество ударений и числительных.
- [] 9. Опробовать Google AI Studio TTS: сравнить чтение по стилям; протестировать multi-speaker на коротком диалоге; зафиксировать квоты и задержки.
- [] 10. Сравнить TTS-провайдеров (включая ElevenLabs) по качеству/консистентности/стоимости; рассчитать бюджеты проектов; проверить разделы Pricing и актуальные тарифы.
- [] 11. Исследовать/пилотировать клонирование голоса: оценить качество при 3 с, 10 с и 3 мин; изучить требования к верификации и правовые риски.
- [] 12. Собрать прототип ассистента на конвейере ASR→LLM→TTS (или на WAPI): измерить латентность, стоимость/мин, надежность; затем протестировать live/end-to-end модель с оценкой латентности и поведения при перебивании.
- [] 13. Подготовить сравнительный отчет по трем сценариям (классический конвейер, оркестратор, live end-to-end) с метриками качества/латентности/стоимости; запланировать пилот с KPI (средняя латентность <1 с, успешные диалоги >98%).